

Αλγόριθμοι και Πολυπλοκότητα

ΦΡΟΝΤΙΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ (7)

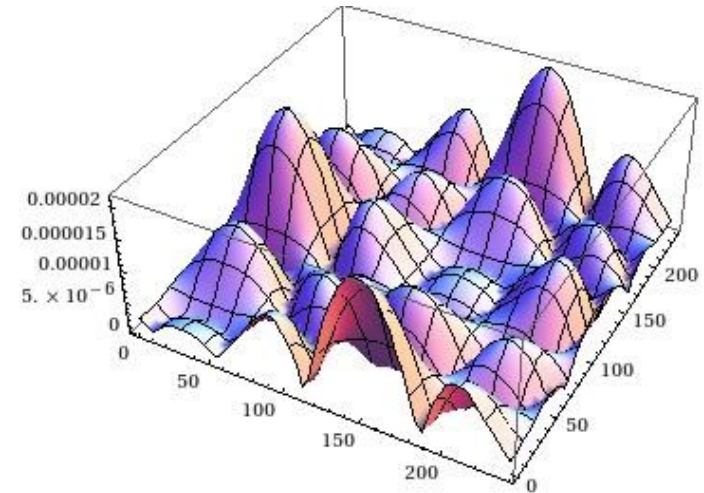
Άπληστοι Αλγόριθμοι 1/2

14/5/2024

Έφη Μαλέσιου
Διδακτορική φοιτήτρια

Άπληστοι αλγόριθμοι

- ποια επιλογή φαίνεται καλύτερη τη δεδομένη στιγμή;
 - επανάληψη σε κάθε βήμα
- αντικειμενική συνάρτηση: όφελος
- σύνολο περιορισμών – εφικτή λύση
- ίδια δεδομένα – διαφορετικός στόχος
 - π.χ. ουρά διοδίων: χρόνος αναμονής ή αριθμός ενεργών ουρών;
 - ελαχιστοποίηση ή μεγιστοποίηση
- τοπικά βέλτιστη επιλογή > καθολικά βέλτιστη λύση;
 - όχι πάντα



Άπληστη επιλογή - βέλτιστη υπο-δομή

- $\exists$ **άπληστοι** αλγόριθμοι που οδηγούν στη **βέλτιστη** λύση του προβλήματος (Prim, Kruskal, Dijkstra)
- **Ιδιότητα άπληστης επιλογής:** η ολικά βέλτιστη λύση είναι δυνατό να παραχθεί κατόπιν μιας και μοναδικής τοπικά (=άπληστα) βέλτιστης επιλογής και προχωρώντας σε επόμενη κίνηση.
 - αγνοούμε τις λύσεις υπο-προβλημάτων, σε αντίθεση με τον δυναμικό προγραμματισμό
- **Βέλτιστη υποδομή:** οι άπληστοι αλγόριθμοι δρουν επαγωγικά και αποδεικνύεται πως η άπληστη επιλογή παρέχει συνολικά βέλτιστη λύση (συνδυαζόμενη με τη βέλτιστη λύση του κάθε μοναδικού υποπροβλήματος)
 - μια βέλτιστη λύση του εμπεριέχει βέλτιστες λύσεις υπο-προβλημάτων
- Έστω ότι **διαφέρει** η άπληστη λύση από τη βέλτιστη $\Rightarrow$ απαγωγή σε **άτοπο**(;)

Δέντρο επικάλυψης

σε συνδεδεμένο, μη-κατευθυνόμενο γράφο με βάρη

ακυκλικό υπο-σύνολο ακμών που περιλαμβάνει όλες τις κορυφές

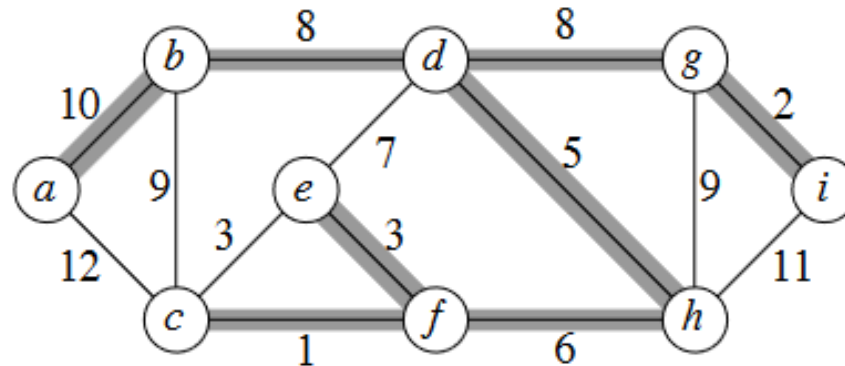
δέντρα = ακυκλικά, μικρότεροι συνεκτικοί υπο-γράφοι τάξης n γράφου G_n
(μοναδικό μονοπάτι μεταξύ ζ. κορυφών)

δεν είναι απαραίτητα μοναδικό για συγκεκριμένο γράφο
έχει $(n-1)$ ακμές

Ανάλογα με τη μοντελοποίηση:

ελάχιστου κόστους = ελάχιστο συνολικό κόστος (π.χ. min. κόστος επισκευών)

μέγιστου κόστους = μέγιστο συνολικό κόστος (π.χ. max. όγκος δεδομένων σε δίκτυο)



δυνατή η αντικατάσταση της (e,f) με την (c,e)
για τη δημιουργία του ΔΕΕΚ (δέντρο
επικάλυψης ελάχιστου κόστους)

Αλγόριθμος Kruskal

σε συνδεδεμένο, μη-κατευθυνόμενο γράφο με βάρη

- σάρωση ακμών κατά αύξουσα σειρά βάρους
- ένωση 2 συνόλων ακμών, επιλέγοντας την ακμή μικρότερου βάρους που τα συνδέει και δε δημιουργεί κύκλο
- $O(V)$: αρχικοποίηση
- $O(E \lg E)$: ταξινόμηση ακμών (π.χ. merge sort)
- $O(E \lg V)$

$$O(E \lg E)$$

$$|E| \leq |V|^2 \Rightarrow \lg |E| = O(2 \lg V) = O(\lg V)$$

KRUSKAL ($G = (V, E, W)$)

1. $T = \emptyset$
 2. **while** ($|T| < n - 1$) **and** ($E \neq \emptyset$) **do**
 3. $e = \text{smallest edge in } E$
 4. $E = E - \{e\}$
 5. **if** $T \cup \{e\}$ has no cycle **then**
 6. $T \leftarrow T \cup \{e\}$
 7. **end if**
 8. **end while**
 9. **if** ($|T| < n - 1$) **then**
 10. write "graph disconnected"
-

Αλγόριθμος Prim

σε συνδεδεμένο, μη-κατευθυνόμενο γράφο με βάρη

- αυθαίρετη ρίζα
- ακμή μικρότερου βάρους που φτάνουν οι κορυφές που έχουμε επισκεφθεί και δεν περιλαμβάνονται ήδη (όχι εισαγωγή κύκλου)
- $O(E \lg V)$
- εξαρτάται από την υλοποίηση

```
PRIM( $V, E, w, r$ )  
 $Q \leftarrow \emptyset$   
for each  $u \in V$   
    do  $key[u] \leftarrow \infty$   
         $\pi[u] \leftarrow \text{NIL}$   
        INSERT( $Q, u$ )  
DECREASE-KEY( $Q, r, 0$ )  $\triangleright key[r] \leftarrow 0$   
while  $Q \neq \emptyset$   
    do  $u \leftarrow \text{EXTRACT-MIN}(Q)$   
        for each  $v \in \text{Adj}[u]$   
            do if  $v \in Q$  and  $w(u, v) < key[v]$   
                then  $\pi[v] \leftarrow u$   
                    DECREASE-KEY( $Q, v, w(u, v)$ )
```



Είναι διαθέσιμος χάρτης με n πόλεις που συνδέονται με ποδηλατικές διαδρομές. Μία διαδρομή που συνδέει δύο πόλεις u, v έχει απόσταση $d(v, u)$. Επιπλέον, το κόστος διανυκτέρευσης στην πόλη v , είναι $c(v)$.

Καλείστε να σχεδιάσετε μία εκδρομή που διαρκεί ακριβώς m ημέρες, ξεκινώντας από την πόλη s και έχοντας ως προορισμό την πόλη t , χωρίς διαμονή στην ίδια πόλη περισσότερες από μία συνεχόμενες ημέρες, και με μέγιστη διανυόμενη απόσταση την ημέρα k , $u(k)$. Επιπλέον, θέλουμε να ελαχιστοποιηθεί το συνολικό κόστος διαμονής.

Προσδιορίστε έναν αλγόριθμο με πολυπλοκότητα $O(n^2(n + m))$ για τον σχεδιασμό εκδρομής σε m πόλεις ($s = v_0, v_1, \dots, t = v_m$) με ελάχιστο κόστος $\sum c(v_i)$, έτσι ώστε $d(v_{i-1}, v_i) \leq u(i)$.

Με την εκδοχή της δυνατής μετάβασης σε περισσότερες από μία πόλεις ανά ημέρα αλλά με 1 πόλη διανυκτέρευσης:

Αρχικά, γίνεται ο υπολογισμός όλων των ζευγών αποστάσεων μεταξύ των πόλεων και αυτές αποθηκεύονται σε έναν πίνακα $d, n \times n$, όπου $d(i, j)$ είναι η απόσταση μεταξύ των πόλεων i, j .

Στη συνέχεια, κατασκευάζεται ένας ακυκλικός (καθώς δεν επιτρέπεται η επιστροφή στην ίδια πόλη και ημέρα) κατευθυνόμενος γράφος, ως εξής:

- Για κάθε πόλη $i \in \{1, 2, \dots, n\}$ και κάθε ημέρα $p \in \{0, 1, \dots, m\}$, προστίθεται μία κορυφή, v_{ip} που συμβολίζει τη διαμονή τη συγκεκριμένη ημέρα στη συγκεκριμένη πόλη. Άρα, οι κορυφές είναι $n(m+1)$.
- Για κάθε ζεύγος πόλεων, i, j όπου $i \neq j$ (αφού η διαμονή στην ίδια πόλη δεν είναι επιτρεπτή για περισσότερες από μία ημέρες) και ημέρα $p \in 1, 2, \dots, m$, προστίθεται η ακμή $(v_{i(p-1)}, v_{jp})$, αν η απόσταση $d(i, j)$ είναι μικρότερη ή ίση της μέγιστης επιτρεπόμενης απόστασης που μπορεί να διανυθεί τη συγκεκριμένη ημέρα, δηλαδή $d(i, j) \leq u(p)$. Άρα, οι ακμές είναι $O(n^2m)$.
- Σε κάθε ακμή $(v_{i(p-1)}, v_{jp})$, ανατίθεται βάρος / κόστος, $c(j)$, που είναι το κόστος διανυκτέρευσης στην πόλη j .

Συνεπώς, η εκδρομή μικρότερου κόστους που διαρκεί ακριβώς m ημέρες, αντιστοιχεί στην εύρεση του μονοπατιού μικρότερου κόστους μεταξύ των κορυφών v_{s0} και v_{tm} . Στην περίπτωση που δε συνδέονται με κάποιο μονοπάτι αυτές οι κορυφές, τότε δεν είναι εφικτή η λύση με τις συγκεκριμένες επιλογές ταξιδιού.

Για αυτόν τον σκοπό μπορεί χρησιμοποιηθεί η τοπολογική ταξινόμηση (προκύπτει άμεσα από τις κορυφές στα διαδοχικά επίπεδα), και στη συνέχεια επισκεπτόμαστε τις κορυφές βάσει της τοπολογικής ταξινόμησης, έχοντας αρχικοποιήσει τις μέγιστες αποστάσεις προς κάθε κόμβο με τιμή $+\infty$. Για κάθε γειτονική κορυφή v της τρέχουσας κορυφής u , ελέγχεται αν η τρέχουσα τιμή του ελάχιστου κόστους έως την v είναι μεγαλύτερη από το άθροισμα του ελάχιστου κόστους έως την κορυφή u με την προσθήκη του βάρους της ακμής (u, v) , περίπτωση κατά την οποία της αποδίδουμε αυτήν την τιμή. Επιπλέον, διατηρούμε σε έναν πίνακα με μία θέση για κάθε κορυφή, το όνομα της κορυφής που τελευταία φορά άλλαξε την τιμή της ελάχιστης απόστασής της, προκειμένου να αναδομηθεί στη συνέχεια το πλήρες μονοπάτι μικρότερου κόστους.

Ως προς την πολυπλοκότητα χρόνου, ο υπολογισμός του πίνακα d , μπορεί να γίνει σε χρόνο $O(n^3)$ με χρήση του αλγορίθμου Dijkstra, 1 φορά για κάθε πόλη από τις n . Ο ακυκλικός κατευθυνόμενος γράφος που σχηματίζεται, έχει $O(n \cdot m)$ πλήθος κορυφών και $O(n^2m)$ πλήθος ακμών και κατά συνέπεια, απαιτείται χρόνος $O(n^2m)$ για την κατασκευή του και $O(n^2m)$ για την εύρεση του συντομότερου μονοπατιού σε αυτόν, από συγκεκριμένη πόλη έναρξης σε συγκεκριμένη πόλη προορισμού. Επιπλέον, σε έναν πίνακα με $m+1$ θέσεις αποθηκεύουμε την κορυφή που τελευταία φορά μείωσε την ελάχιστη τιμή εκάστοτε άλλης και έτσι θα χρειαστεί να τον διατρέξουμε το πολύ μία φορά κατά την οπισθοδρόμηση για τον προσδιορισμό του μονοπατιού μικρότερου κόστους ($O(m)$). Άρα, ο συνολικός χρόνος είναι $\max\{O(m), O(n^3 + n^2m)\} = O(n^3 + n^2m) = O(n^2(n + m))$.

Έστω $G=(V,E)$ ένας απλός συνεκτικός γράφος, του οποίου κάθε ακμή έχει διαφορετικό βάρος.

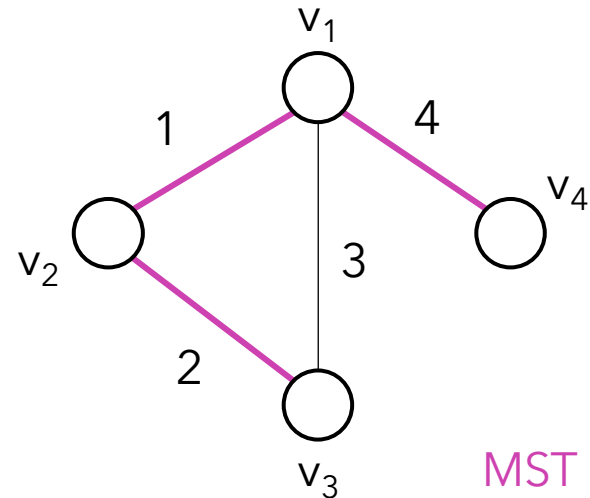
Αποδείξτε αν τα ακόλουθα είναι σωστά ή λαθεμένα;

A. Η ακμή με το **δεύτερο** μικρότερο βάρος ανήκει στο ελάχιστο δέντρο επικάλυψης (MST) του G .

B. Η ακμή με το **τρίτο** μικρότερο βάρος ανήκει στο ελάχιστο δέντρο επικάλυψης (MST) του G .

A. Είναι σωστό. Σε έναν γράφο με διαφορετικές τιμές βαρών για κάθε ακμή, το MST είναι μοναδικό. Ο αλγόριθμος Kruskal, ο οποίος επιστρέφει τη βέλτιστη λύση κάθε φορά, εξετάζει τις ακμές με αύξουσα σειρά των βαρών τους. Έτσι, είναι δεδομένο πως αρχικά θα επιλεγεί αυτή με το ελάχιστο βάρος. Επιπλέον, στην περίπτωση αυτού του G , θα επιλέξει σίγουρα και την ακμή με το δεύτερο μεγαλύτερο βάρος γιατί ο γράφος είναι απλός και κατά συνέπεια δεν είναι δυνατό να δημιουργηθεί κύκλος με 3 κορυφές και 2 ακμές (κριτήριο μη-επιλογής της ακμής).

B. Είναι λαθεμένο, καθώς υπάρχει το ακόλουθο αντιπαράδειγμα.



Δίνεται ένας πίνακας A με n αριθμούς και ζητείται να βρεθεί σε χρόνο $O(n)$, η υπακολουθία μέγιστου μήκους με την ιδιότητα:

$A[i1] > A[i2], A[i2] < A[i3], A[i3] > A[i4], A[i4] < A[i5], \dots$ κ.ο.κ.

Η υπακολουθία δε χρειάζεται να βρίσκεται σε συνεχόμενες θέσεις.

Θα χρησιμοποιήσουμε έναν άπληστο αλγόριθμο που επιλέγει τις «κορυφές» και τις «κοιλιάδες» του πίνακα A. Ας υποθέσουμε ότι ο πίνακας A είναι ο [4,10,12,9,3,1,0,6,5,4,3,8,10,15]. Αποσυνθέτουμε τον πίνακα σε εναλλασσόμενες φθίνουσες και αύξουσες ακολουθίες μέγιστης δυνατής έκτασης. Στον πίνακα του παραδείγματος, έχουμε αρχικά την αύξουσα ακολουθία (4,10,12) που ακολουθείται από τη φθίνουσα ακολουθία (12,9,3,1,0) και στη συνέχεια την αύξουσα ακολουθία (0,6), τη φθίνουσα (6,5,4,3) και τελικά την αύξουσα (3,8,10,15). Σημειώνεται ότι αυτές οι ακολουθίες επικαλύπτονται στα άκρα τους.

Ο άπληστος αλγόριθμος, επιλέγει τα τελικά στοιχεία καθεμίας από αυτές τις ακολουθίες, το οποίο οδηγεί ξεκάθαρα σε υπακολουθία με την επιθυμητή ιδιότητα. Επιπλέον, απαιτεί τη σάρωση του πίνακα μόνο μία φορά για κάθε θέση και έτσι έχει πολυπλοκότητα χρόνου $O(n)$.

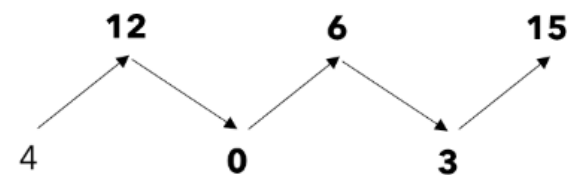
Αρκεί να δειχθεί ότι είναι και η υπακολουθία μέγιστου μήκους με την επιθυμητή ιδιότητα.

Με δεδομένη την αποσύνθεση του πίνακα σε αύξουσες / φθίνουσες ακολουθίες, όπως αναφέρθηκε παραπάνω, εστιάζουμε σε μία από αυτές τις ακολουθίες, $A[i,...,j]$. Έστω S η βέλτιστη λύση.

Τότε, μόνο 1 ή 2 στοιχεία από την ακολουθία $A[i,...,j]$ θα μπορούσαν να ανήκουν στην S, διαφορετικά 3 ή περισσότερα διαδοχικά στοιχεία θα οδηγούσαν στον σχηματισμό αύξουσας ή φθίνουσας ακολουθίας και η S δε θα είχε την επιθυμητή ιδιότητα.

4	10	12	9	3	1	0	6	5	4	3	8	10	15
---	----	----	---	---	---	---	---	---	---	---	---	----	----

4	12	0	6	3	15
---	----	---	---	---	----



12 > 0, 0 < 6, 6 > 3, 3 < 15
12, 0, 6, 3, 15

Ο Γιώργος δουλεύει σε ένα καθαριστήριο ρούχων, και κάθε πρωί πρέπει να ελέγξει τα ρούχα για λεκέδες, προκειμένου να τα επεξεργαστεί κατάλληλα, και στη συνέχεια να τα τοποθετήσει στο πλυντήριο και στο στεγνωτήριο. Ο Γιώργος, μπορεί να αντιμετωπίζει τους λεκέδες σε ένα ρούχο κάθε φορά και κάθε είδος απαιτεί διαφορετικό χρόνο ελέγχου και αντιμετώπισης. Ωστόσο, μπορούν να πλένονται και να στεγνώνουν ταυτόχρονα. Ο Γιώργος θέλει να τελειώσει γρήγορα τη δουλειά, οπότε ψάχνει να βρει ποια είναι η καλύτερη σειρά ελέγχου των ρούχων.

Ζητείται να δοθεί αποδοτικός αλγόριθμος που θα δίνει ένα χρονοδιάγραμμα του οποίου ο χρόνος ολοκλήρωσης είναι όσο το δυνατό μικρότερος.

Έστω ότι έχουμε n ρούχα, τα οποία αριθμούμε $1, 2, \dots, n$ και έστω ότι συμβολίζουμε με:

- s_i , τον προβλεπόμενο χρόνο του ρούχου i για έλεγχο / αντιμετώπιση των λεκέδων
- r_i , τον προβλεπόμενο χρόνο του ρούχου i για να πλυθεί
- b_i , τον προβλεπόμενο χρόνο του ρούχου i για να στεγνώσει

Ο άπληστος αλγόριθμος που μπορεί να δρομολογήσει τη σειρά για τον έλεγχο των λεκέδων, ώστε να ολοκληρωθεί όλη η διαδικασία όσο το δυνατό πιο σύντομα, είναι ο εξής: διατάσσουμε τα ρούχα κατά φθίνουσα σειρά $(r_i + b_i)$, και οδηγούνται για πλύσιμο και στέγνωμα με αυτήν τη σειρά, δηλαδή πρώτο θα επεξεργαστεί ο Γιώργος το ρούχο που χρειάζεται τον περισσότερο χρόνο για να πλυθεί και να στεγνώσει. Η ταξινόμηση των n ρούχων βάσει του αθροίσματος αυτού, απαιτεί χρόνο $O(n \log n)$.

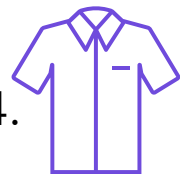
Μάλιστα, αυτός ο αλγόριθμος οδηγεί στη βέλτιστη λύση, δηλαδή ελαχιστοποιεί τον χρόνο ολοκλήρωσης της διαδικασίας.

- Έστω ότι έχουμε μια οποιαδήποτε βέλτιστη λύση η οποία δε χρησιμοποιεί αυτήν τη διάταξη.
- Τότε, η βέλτιστη λύση θα έχει 2 ρούχα i και j , τέτοια ώστε το j να ξεκινάει τη διαδικασία αμέσως μετά το i με $r_i + b_i < r_j + b_j$.
- Ορίζεται το ζεύγος (i, j) ως αντιστροφή.

- Σε αυτήν την ανεστραμμένη δρομολόγηση, η επεξεργασία του j θα ξεκινήσει μετά την επεξεργασία του i :
 - για το i θα τελειώσει η επεξεργασία νωρίτερα από ό,τι θα τελείωνε πριν
 - για το j θα τελειώσει η επεξεργασία αφού πρώτα τελειώσει η επεξεργασία για το i
- Όμως, αφού ισχύει $r_i + b_i < r_j + b_j$, αυτό σημαίνει ότι έτσι μετατίθεται η επεξεργασία ρούχου που απαιτεί περισσότερο χρόνο σε μεταγενέστερη στιγμή. Επομένως, δε θα επιφέρει μείωση του συνολικού χρόνου.

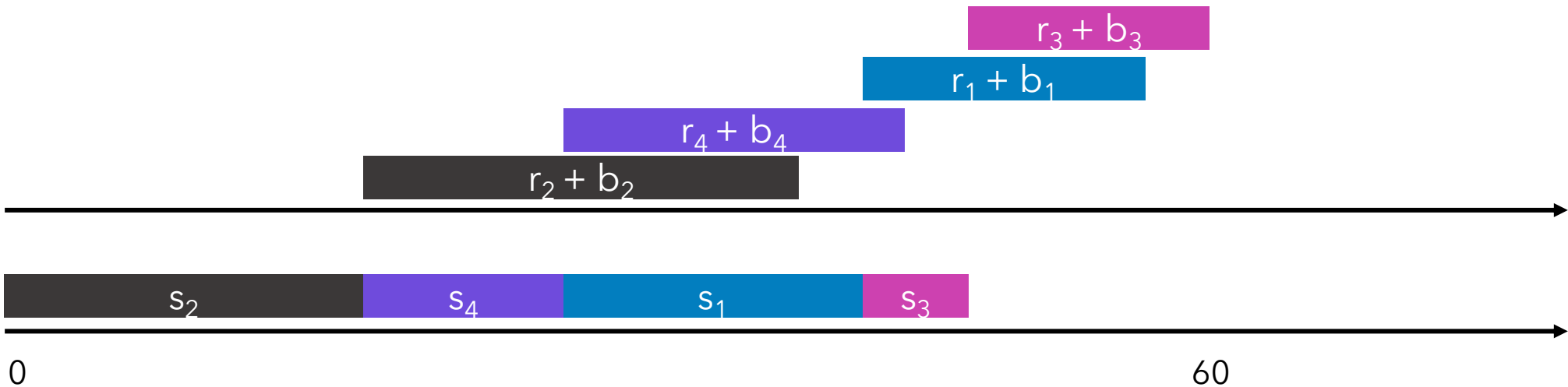
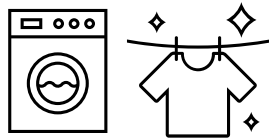
Συνεχίζοντας με τον ίδιο τρόπο, μπορούμε να απαλείψουμε όλες τις αντιστροφές χωρίς αύξηση του χρόνου ολοκλήρωσης. Επομένως, στο τέλος αυτής της διαδικασίας, θα έχουμε μια δρομολόγηση με τη σειρά που επιλέγεται από τον άπληστο αλγόριθμο, όπου ο χρόνος ολοκλήρωσης δεν είναι μεγαλύτερος από αυτόν της βέλτιστης λύσης. Άρα, ο άπληστος αλγόριθμος είναι βέλτιστος.



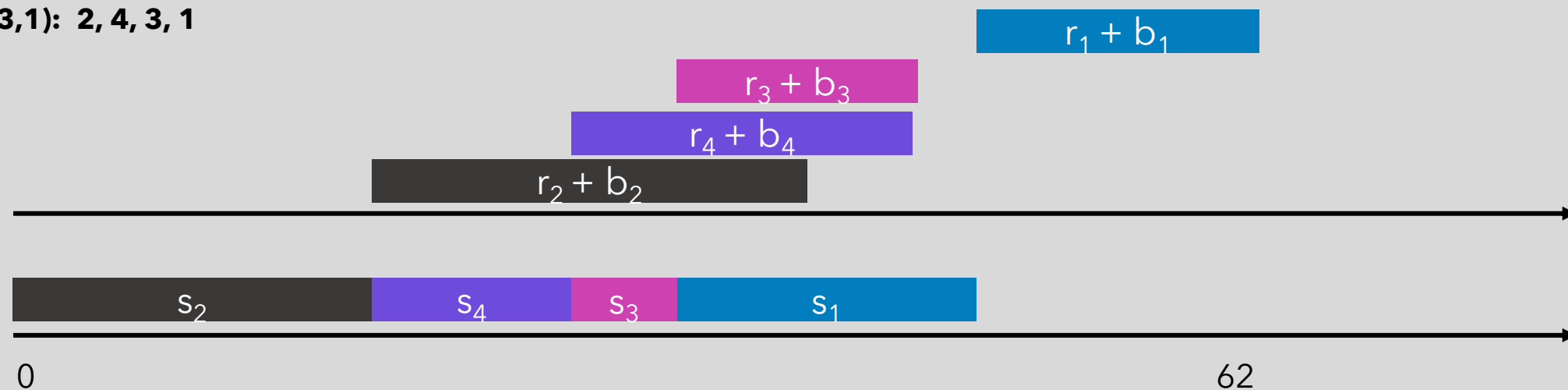
	s_i στη σειρά	r_i παράλληλα	b_i
1. 	15	8	6
2. 	18	12	20
3. 	5	7	5
4. 	10	7	10

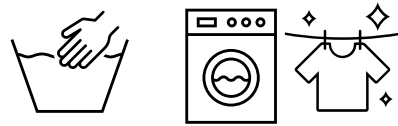
φθίνουσα σειρά ($r_i + b_i$): 2, 4, 1, 3

φθίνουσα σειρά ($r_i + b_i$): 2, 4, 1, 3



με μία αντιστροφή (3,1): 2, 4, 3, 1





	s_i στη σειρά	r_i παράλληλα	b_i
1. 	10	10	4
2. 	1000	2	2

φθίνουσα σειρά ($r_i + b_i$): 1, 2

- $s1 + s2 = 1010$, στη σειρά
- $(r1+s1)$, ολοκλήρωση πριν την ολοκλήρωση του $s2$
- $(r2+s2)$, μετά την ολοκλήρωση του $s2$
- συνολικός χρόνος = 1014

με μία αντιστροφή (2,1): 2, 1

- $s2 + s1 = 1010$, στη σειρά
- $(r2+s2)$, ολοκλήρωση πριν την ολοκλήρωση του $s1$
- $(r1+s1)$, μετά την ολοκλήρωση του $s1$
- συνολικός χρόνος = 1024

Θέλουμε να δώσουμε ρέστα, n cents,
χρησιμοποιώντας τον ελάχιστο αριθμό νομισμάτων,
από τα νομίσματα αξίας: 1, 5, 10, 25 cents

Άπληστος αλγόριθμος που οδηγεί σε βέλτιστη λύση;



Με άπληστο τρόπο, σε κάθε βήμα επιλέγουμε να δώσουμε το νόμισμα μεγαλύτερης δυνατής αξίας έως τη συμπλήρωση των n cents.

Έστω ότι $G = (g_1, g_2, \dots, g_k)$ είναι η λύση του προβλήματος που προκύπτει με τον άπληστο αλγόριθμο και $O = (o_1, o_2, \dots, o_l)$ είναι η βέλτιστη λύση, όπου τα νομίσματα κάθε λύσης είναι ταξινομημένα με φθίνουσα σειρά αξίας. Παρατηρείται ότι στη βέλτιστη λύση, πρέπει να υπάρχουν:

- το πολύ 4 νομίσματα αξίας 1, αφού 5 τέτοια νομίσματα μπορούν να αντικατασταθούν από 1 αξίας 5
- το πολύ 1 νόμισμα αξίας 5, αφού 2 τέτοια νομίσματα μπορούν να αντικατασταθούν από 1 αξίας 10
- το πολύ 2 νομίσματα αξίας 10, αφού 3 τέτοια νομίσματα μπορούν να αντικατασταθούν από 1 αξίας 25 και 1 αξίας 5

Αν υποθέσουμε ότι οι λύσεις G και O διαφέρουν και η πρώτη διαφορά τους εντοπίζεται στη θέση i , τότε θα πρέπει $g_i > o_i$, αφού ο άπληστος αλγόριθμος επιλέγει πάντοτε το νόμισμα με τη μεγαλύτερη δυνατή αξία που δεν ξεπερνά το εκάστοτε υπολειπόμενο ποσό. Συνεπώς, υπάρχουν 3 περιπτώσεις:

- Αν $g_i = 25$: Τότε, τα μετέπειτα νομίσματα της βέλτιστης λύσης, θα πρέπει να έχουν συνολική αξία ≥ 25 . Από την αρχική παρατήρηση για το μέγιστο δυνατό πλήθος των 3 νομισμάτων μικρότερης αξίας, προκύπτει ότι τα νομίσματα μπορούν να είναι μόνο 2 αξίας 10 λεπτών και 1 αξίας 5, τα οποία όμως μπορούν να αντικατασταθούν από ένα και άρα να έχουμε λύση με λιγότερα νομίσματα, που αποτελεί αντίφαση για την υπόθεση της βέλτιστης λύσης.

- Αν $g_i = 10$: Τότε, τα μετέπειτα νομίσματα της βέλτιστης λύσης, μπορούν να είναι μόνο 1 αξίας 5 και 4 αξίας 1 το πολύ, αλλά με αυτά έχουμε μέγιστο άθροισμα αξιών 9 και όχι 10 και έτσι η βέλτιστη λύση δε θα μπορούσε να είναι έγκυρη, οδηγώντας πάλι σε αντίφαση.
- Αν $g_i = 5$: Τότε, τα μετέπειτα νομίσματα της βέλτιστης λύσης, μπορούν να είναι μόνο 4 αξίας 1 το πολύ, αλλά με αυτά έχουμε μέγιστο άθροισμα αξιών 4 και όχι 5 και έτσι η βέλτιστη λύση δε θα μπορούσε να είναι έγκυρη, οδηγώντας πάλι σε αντίφαση.

Συνεπώς, δεν μπορούν να διαφέρουν οι δύο αυτές λύσεις και άρα η λύση του άπληστου αλγορίθμου είναι και βέλτιστη.

Επισημαίνεται ότι αυτό δεν ισχύει για κάθε περίπτωση νομισμάτων: π.χ. αν είχαμε νομίσματα αξίας 1, 10 και 15 λεπτών και έπρεπε να δώσουμε ρέστα 20 λεπτών, ο άπληστος αλγόριθμος θα επέστρεφε (15,1,1,1,1,1) όμως η βέλτιστη είναι (10,10).

Αλγόριθμος 3 COINS(x)

Είσοδος: Δίνεται ένα ποσό σε x σε cents.

Έξοδος: Το μικρότερο πλήθος κερμάτων που μπορούμε να χαλάσουμε το ποσό x .

1: $coin2[1] = 25, coin2[2] = 10, coin2[3] = 5, coin2[4] = 1$

2: **for** $i = 1$ to 4 **do**

3: $coin[i] = 0$

4: **end for**

5: $i = 1$

6: **while** $i \leq 4$ AND $x > 0$ **do**

7: **while** $x \geq coin2[i]$ **do**

8: $coin[i] = coin[i] + 1$

9: $x = x - coin2[i]$

10: **end while**

11: $i = i + 1$

12: **end while**

Ο καθηγητής Μίδας οδηγεί με αυτοκίνητο από τον Πύργο στην Αλεξανδρούπολη. Η δεξαμενή καυσίμων του αυτοκινήτου του, όταν είναι γεμάτη, έχει αρκετά καύσιμα ώστε να οδηγεί για n χιλιόμετρα και ο χάρτης του, δείχνει τις αποστάσεις μεταξύ των σταθμών καυσίμων στο δρόμο του. Ο καθηγητής θέλει να κάνει όσες λιγότερες στάσεις γίνεται. Δώστε έναν αποδοτικό (άπληστο) αλγόριθμο με τον οποίο ο καθηγητής Μίδας θα προσδιορίζει σε ποιούς σταθμούς πρέπει να κάνει στάση και αποδείξτε ότι ο αλγόριθμος σας δίνει μια βέλτιστη λύση.

Ξεκινώντας με γεμάτη δεξαμενή καυσίμων, ο καθηγητής Μίδας οδηγεί μέχρι τον πιο μακρινό σταθμό καυσίμων που βρίσκεται στα επόμενα n χιλιόμετρα από την αφετηρία (Πύργος) και γεμίζει τη δεξαμενή καυσίμων του εκεί. Έπειτα πηγαίνει στον πιο μακρινό σταθμό που βρίσκεται στα επόμενα n χιλιόμετρα από την τελευταία του στάση και ξαναγεμίζει την δεξαμενή καυσίμων του. Συνεχίζει έτσι.

Βλέποντας το πρόβλημα διαφορετικά, σε κάθε σταθμό καυσίμων, ο καθηγητής Μίδας πρέπει να ελέγχει αν μπορεί να φτάσει μέχρι τον επόμενο σταθμό ή αν θα σταματήσει σε αυτόν. Αν μπορεί να φτάσει στον επόμενο σταθμό δεν σταματάει σ' αυτόν. Αν δε μπορεί τότε σταματάει και γεμίζει τη δεξαμενή καυσίμων του. Ο καθηγητής Μίδας δε χρειάζεται να γνωρίζει πόσα καύσιμα έχει ή πόσο μακριά είναι ο επόμενος σταθμός καυσίμων για να υλοποιήσει την παραπάνω προσέγγιση αφού σε κάθε στάση (και συνεπώς γέμισμα της δεξαμενής καυσίμων) μπορεί να προσδιορίσει ποιός είναι ο επόμενος σταθμός που πρέπει να σταματήσει.

Ο αλγόριθμος είναι βέλτιστος. Ας υποθέσουμε ότι υπάρχουν m σταθμοί καυσίμων. Θεωρούμε βέλτιστη λύση με s σταθμούς στους οποίους κάνει στάση ο καθηγητής Μίδας και η πρώτη στάση είναι στον k σταθμό καυσίμων. Τότε η υπόλοιπη βέλτιστη λύση πρέπει να είναι βέλτιστη στο υποπρόβλημα με τους υπόλοιπους $m - k$ σταθμούς. Αλλιώς, αν υπήρχε καλύτερη λύση για το υποπρόβλημα δηλαδή, λύση με λιγότερες από $s - 1$ στάσεις, θα μπορούσαμε να το χρησιμοποιήσουμε για να βρούμε μια λύση με λιγότερες από s στάσεις για το αρχικό πρόβλημα, αντίφαση με την αρχική υπόθεση ότι έχουμε βέλτιστη λύση με s σταθμούς.

Ο αλγόριθμός είναι άπλητος. Ας υποθέσουμε ότι υπάρχουν k σταθμοί καυσίμων από την αρχή μέχρι το n -οστό χιλιόμετρο. Η άπληστη λύση επιλέγει τον k σταθμό ως τη πρώτη στάση που θα κάνει ο καθηγητής Μίδας. Κανένας σταθμός μετά τον k δε μπορεί να είναι ο πρώτος σταθμός που θα κάνει στάση ο καθηγητής, αφού θα έχει ξεμείνει από καύσιμα. Αν μια λύση επιλέξει τον σταθμό $j < k$ ως πρώτη στάση τότε, ο καθηγητής Μίδας θα μπορούσε να επιλέξει το σταθμό k αντί για τον j και τότε θα είχε περισσότερα καύσιμα φεύγοντας από τον k από ότι θα είχε αν σταματούσε στον σταθμό j . Έτσι, θα έφτανε πιο μακριά χωρίς να γεμίσει τη δεξαμενή καυσίμων αν επιλέξει να σταματήσει στον σταθμό k .

Αν υπάρχουν m σταθμοί καυσίμων στο χάρτη, ο Μίδας χρειάζεται να ελέγξει κάθε σταθμό μία φορά. Η πολυπλοκότητα $O(m)$.

α) Δείξτε τα διαδοχικά βήματα κατασκευής του δένδρου Huffman για την κωδικοποίηση των χαρακτήρων A (0,31), N(0,24), T(0,20), K(0,15), Σ(0,10) (στις παρενθέσεις δίνονται οι συχνότητές τους). (5 μονάδες)

β) Δώστε τον πίνακα κωδικοποίησης των χαρακτήρων (5 μονάδες)

γ) Κωδικοποιήστε το «κείμενο» ΚΑΣΤΑΝΑΣ (5 μονάδες)

δ) Αποκωδικοποιήστε το «μήνυμα» 0100100101101 αγνοώντας τυχόν υπόλοιπο του μηνύματος (5 μονάδες)

- **άπληστος:** διατάσσουμε τους κόμβους βάσει της συχνότητας και σε κάθε βήμα επιλέγουμε να ενώσουμε τους 2 κόμβους με τις μικρότερες τιμές (πρώτα φτιάχνουμε κόμβους σε μεγαλύτερο βάθος ώστε να αναπαρίστανται με περισσότερα bits οι λιγότερο συχνοί χαρακτήρες)
- **βέλτιστος:** δημιουργία πλήρους δυαδικού δέντρου

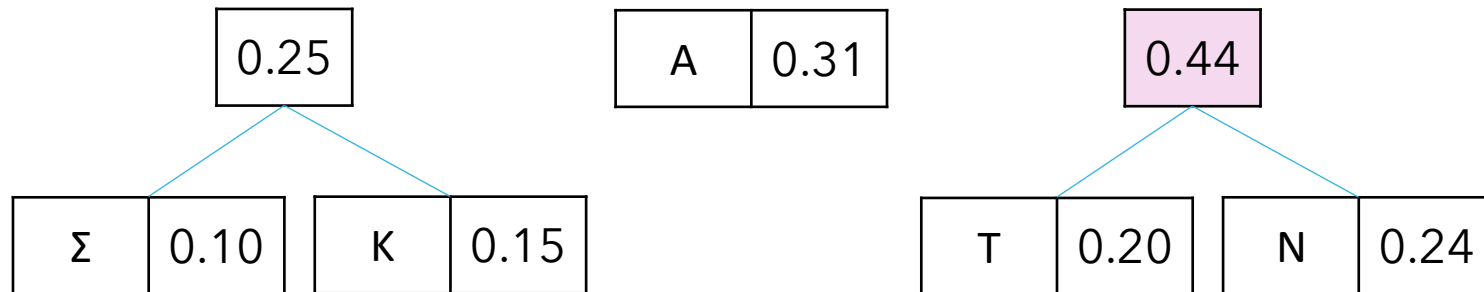
Οι κόμβοι σε αύξουσα διάταξη συχνότητας:

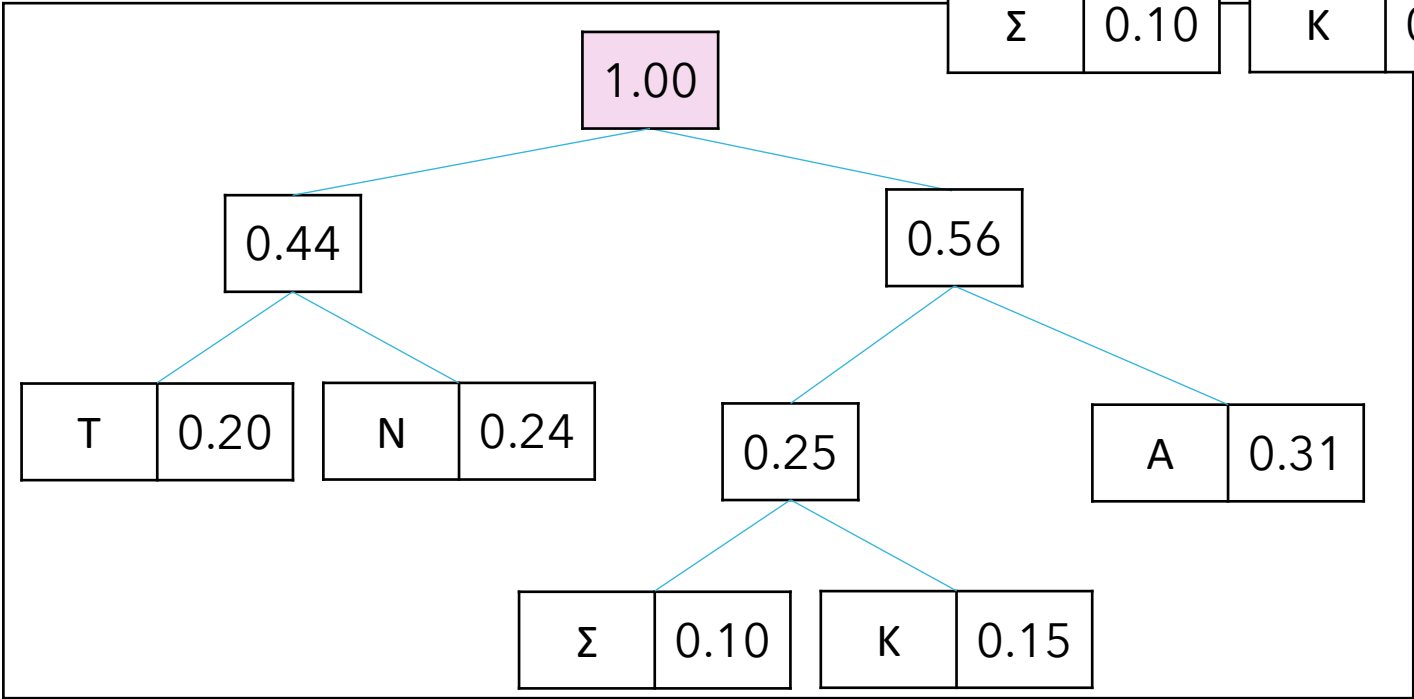
A	0.31
---	------

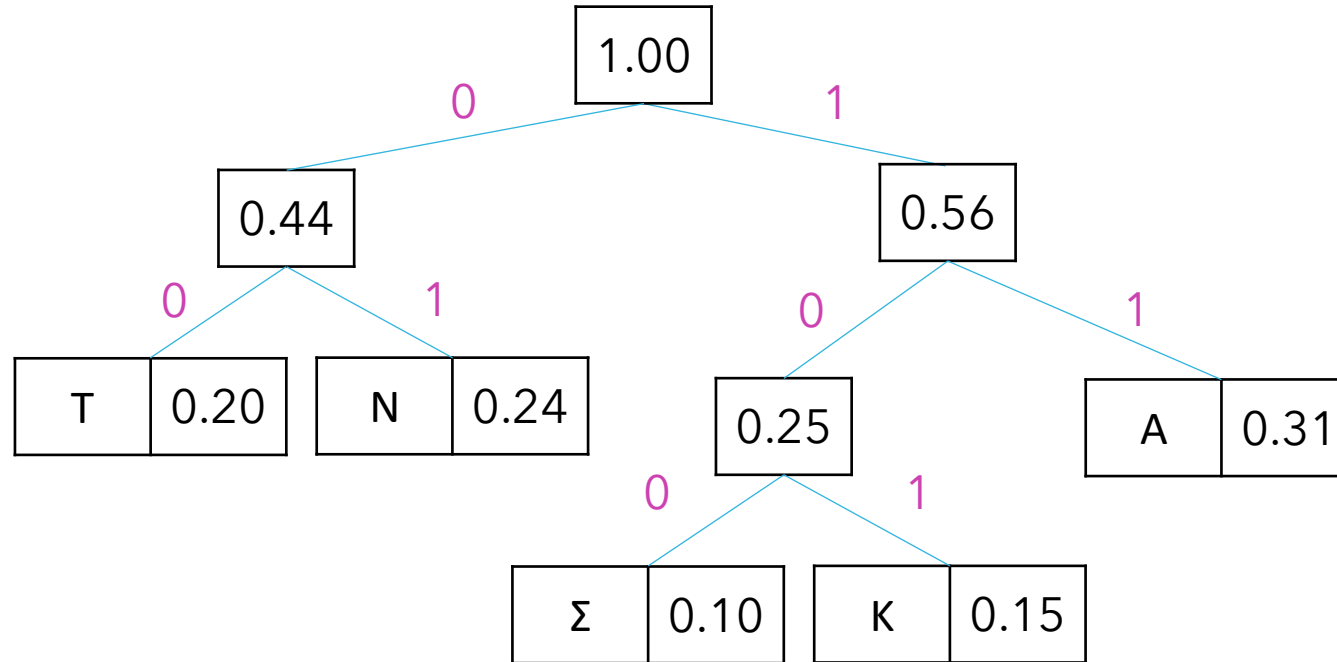
Οι δύο κόμβοι με μικρότερη συχνότητας, συγχωνεύονται:

A	0.31
---	------

K	0.15
---	------







Κωδικοποίηση:

A = 11

N = 01

T = 00

K = 101

Σ = 100

K A Σ T A N A Σ
101 11 100 00 11 01 11 100

0100100101101

N T Σ K K

παρατηρείται ότι καμία κωδικοποίηση χαρακτήρα δεν αποτελεί πρόθεμα της κωδικοποίησης οποιουδήποτε άλλου

Κωδικοποίηση 100 χαρακτήρων:

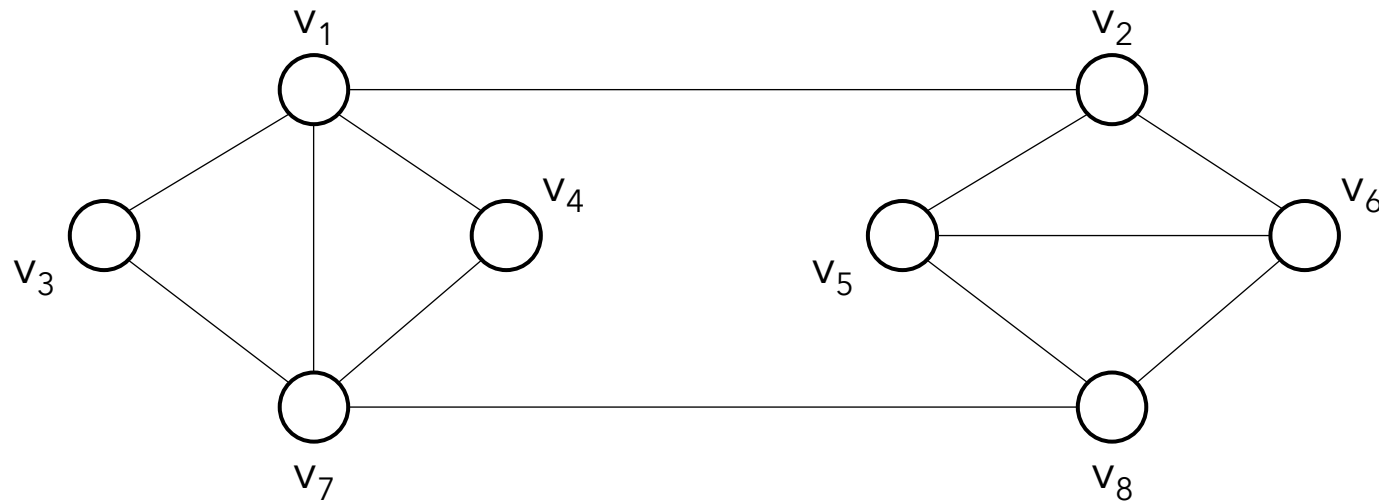
Naive Coding [\[Hide\]](#)

11010110110101101101011011010110110101101101011011010110110101101101011011010110110
101101101011011010110110101101000011010000110100001101000011010000110100001101000011010000
11010000110100001101000011010000110100001101000011010000110100001101000011010000110100001
10100001101000011010000110100001101000011010000110100001101000011010000110100001101000011
01000011011001110110011101100111011001110110011101100111011001110110011101100111011001110001
0111000101110001011100010111000101110001011100010111000101110001011100010111000101110001011
1000101110001011100010111000101110001011100010111000101110001011100010111000101110011101100
111011001110110011101100111011001110110011101100111011001110110011101100111011001110110011101
1001110110011101100111011001110110011101100111011001110110011101100111011001110110

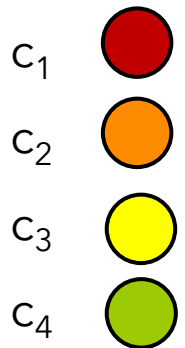
Huffman Coding [\[Hide\]](#)

[illegible]

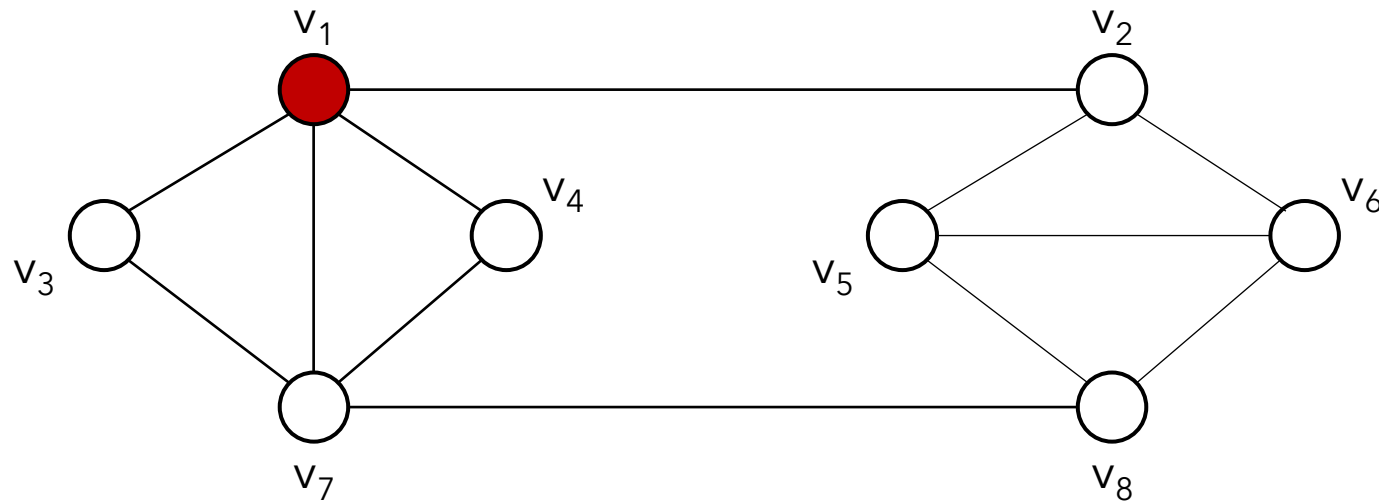
Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα



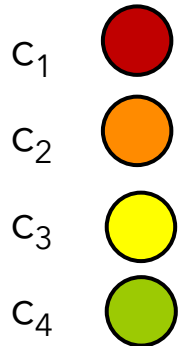
Τυχαία διάταξη κορυφών: $(v_1, v_7, v_3, v_4, v_2, v_6, v_5, v_8)$
Οι κορυφές εξετάζονται με τη σειρά και αποδίδεται το πρώτο χρώμα που είναι δυνατό



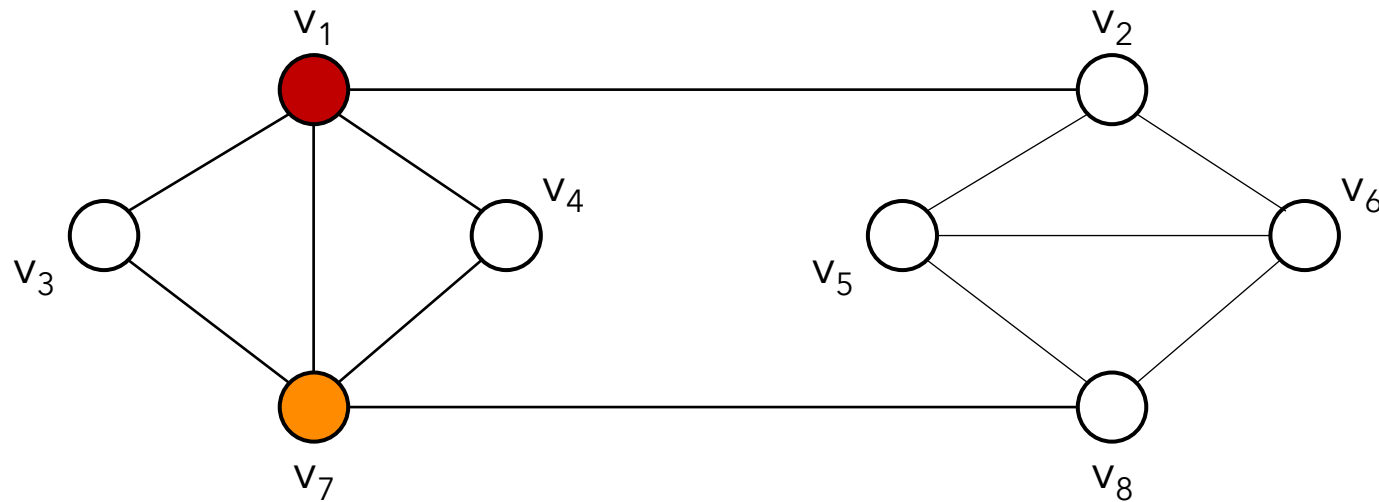
Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα



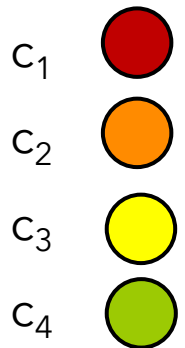
Τυχαία διάταξη κορυφών: ($\mathbf{v_1}$, v_7 , v_3 , v_4 , v_2 , v_6 , v_5 , v_8)



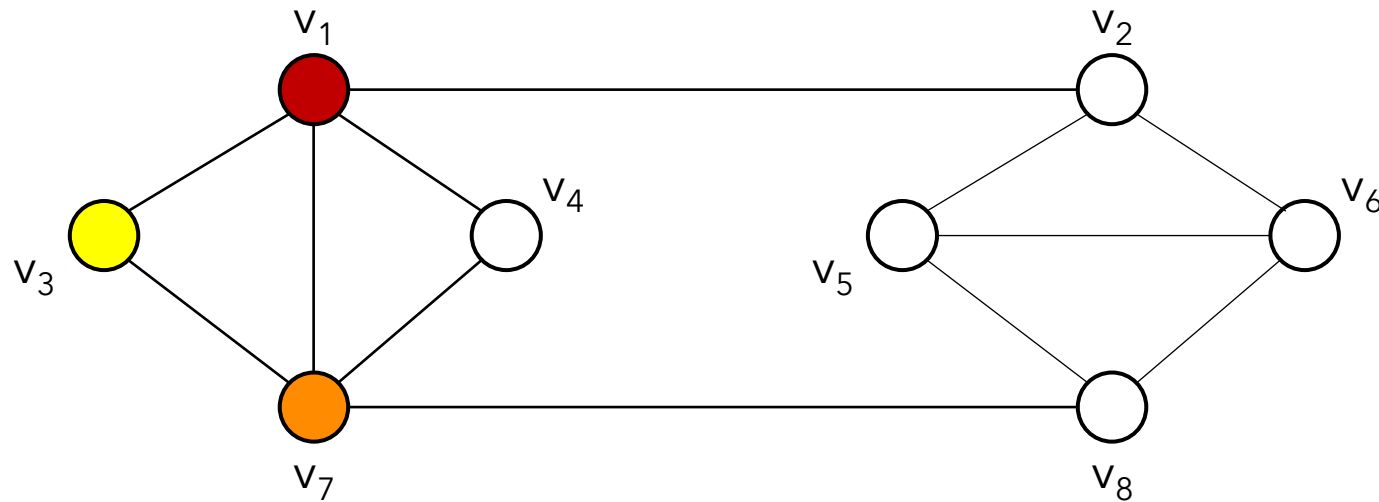
Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα



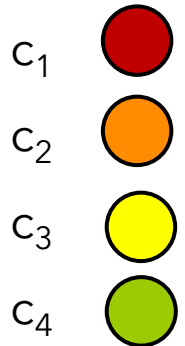
Τυχαία διάταξη κορυφών: ($v_1, v_7, v_3, v_4, v_2, v_6, v_5, v_8$)



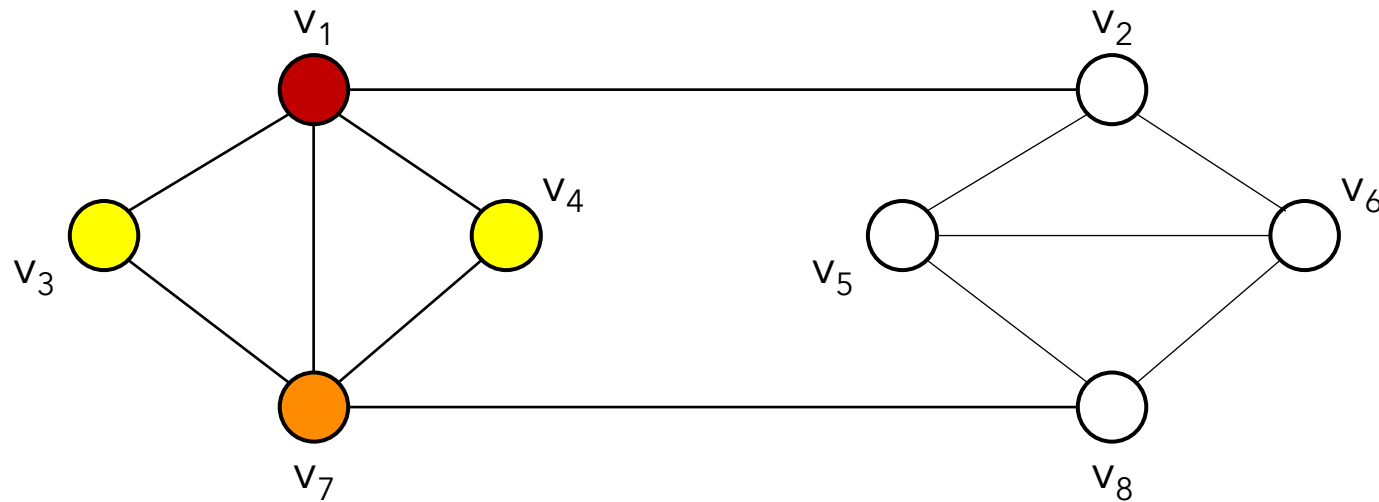
Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα



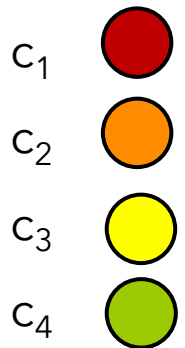
Τυχαία διάταξη κορυφών: ($\mathbf{v_1}$, $\mathbf{v_7}$, $\mathbf{v_3}$, v_4 , v_2 , v_6 , v_5 , v_8)



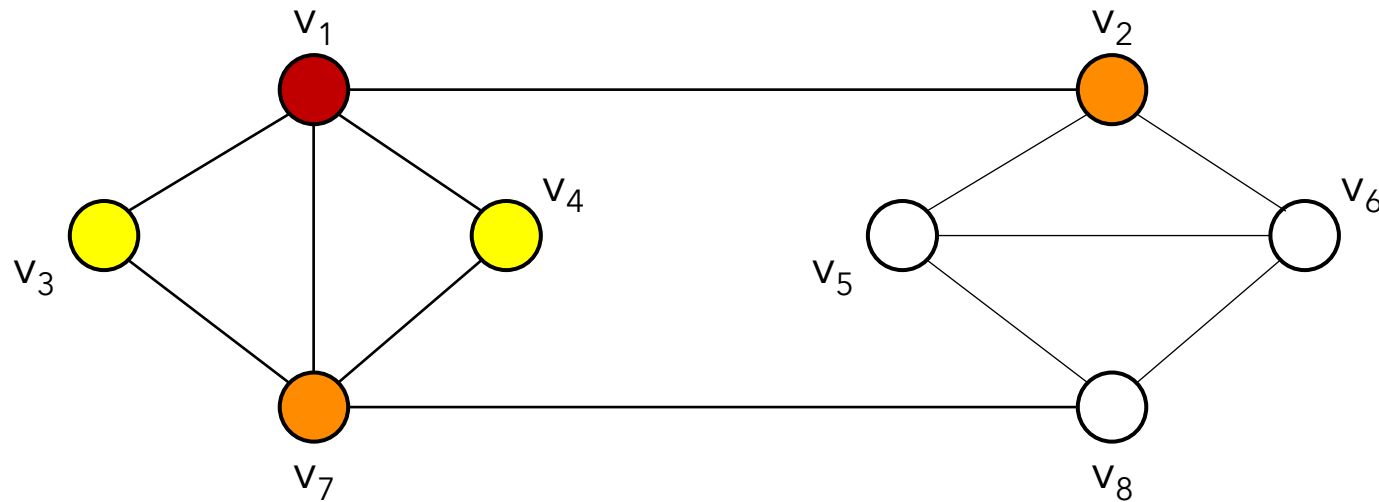
Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα



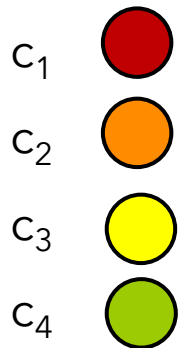
Τυχαία διάταξη κορυφών: (v_1 , v_7 , v_3 , v_4 , v_2 , v_6 , v_5 , v_8)



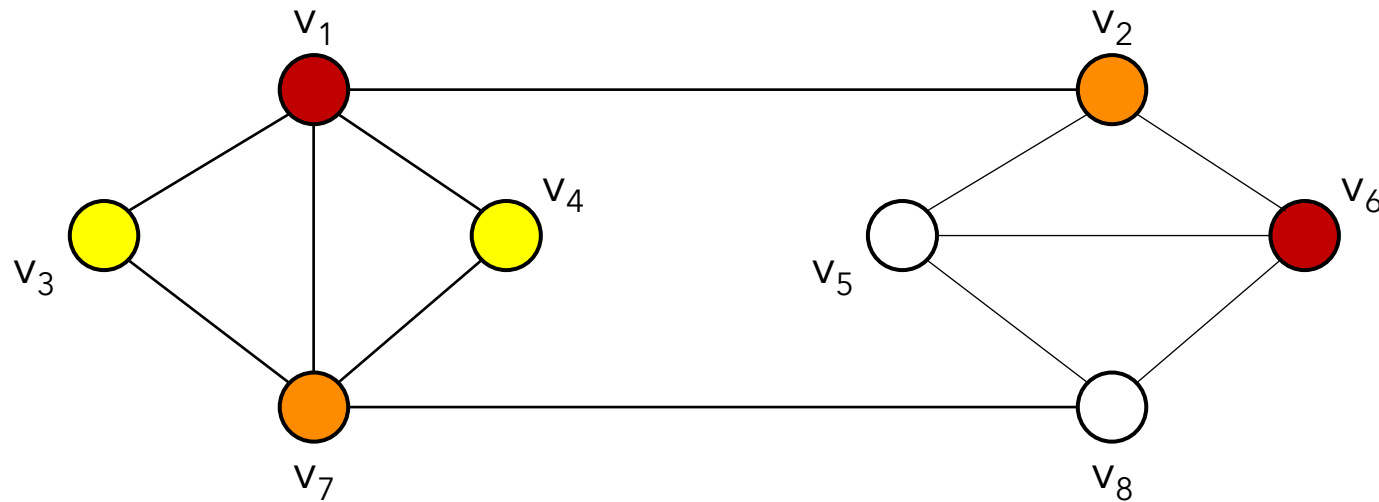
Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα



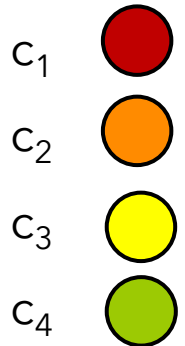
Τυχαία διάταξη κορυφών: ($v_1, v_7, v_3, v_4, v_2, v_6, v_5, v_8$)



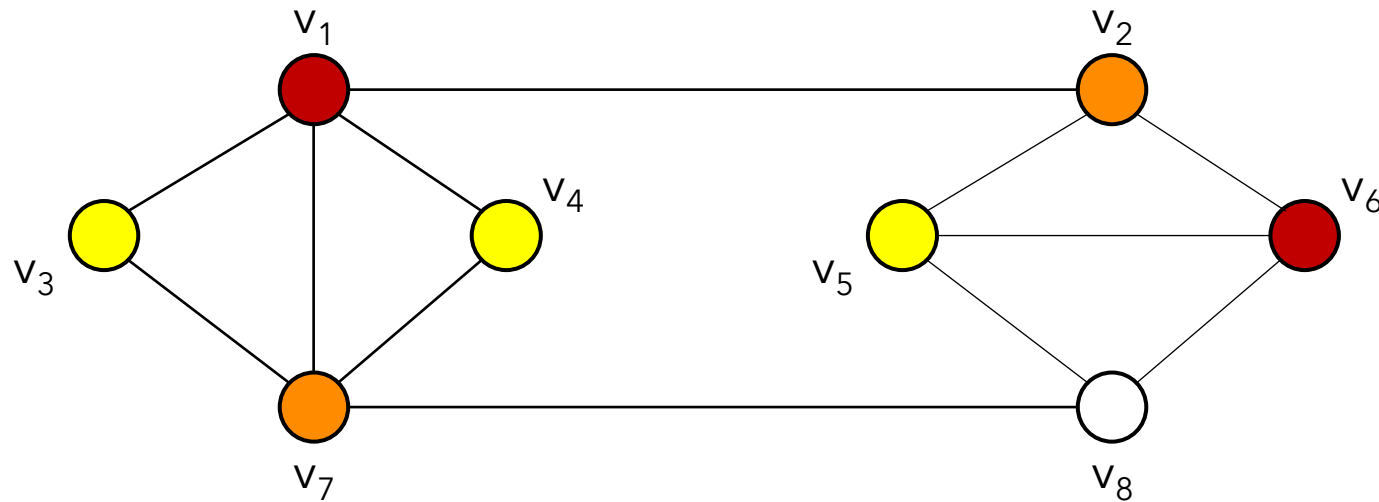
Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα



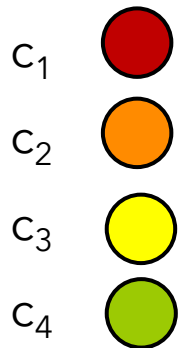
Τυχαία διάταξη κορυφών: ($v_1, v_7, v_3, v_4, v_2, v_6, v_5, v_8$)



Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα

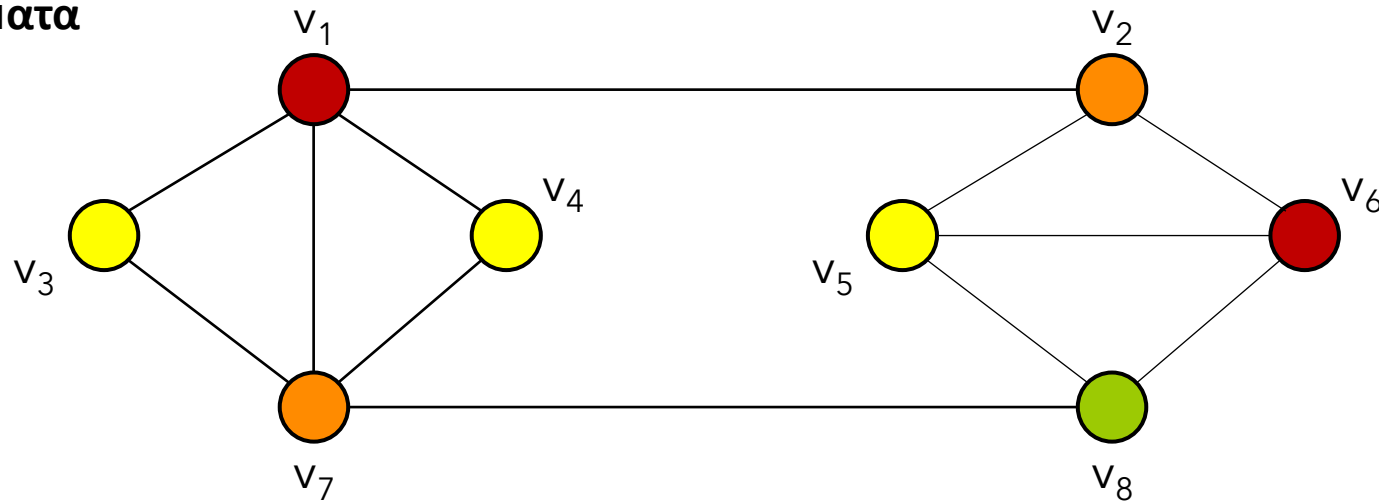


Τυχαία διάταξη κορυφών: (**v₁**, **v₇**, **v₃**, **v₄**, **v₂**, **v₆**, **v₅**, v₈)

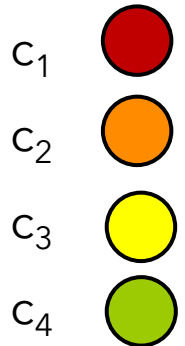


Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα

Χρειάστηκαν **4** χρώματα



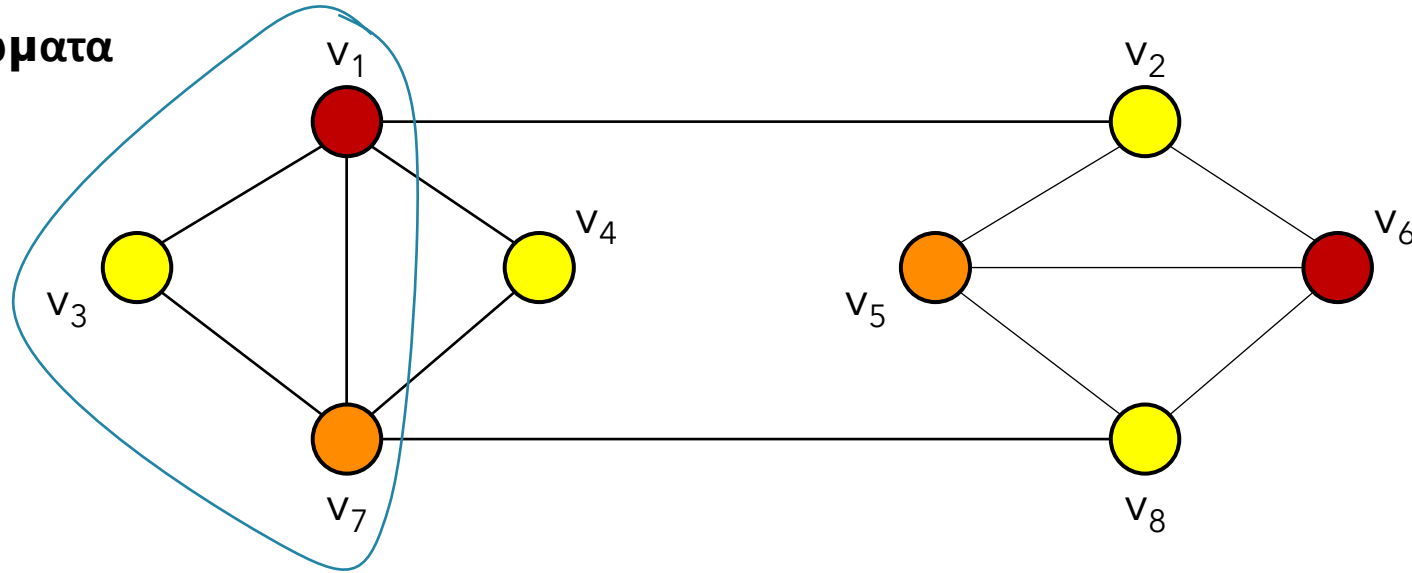
Τυχαία διάταξη κορυφών: ($v_1, v_7, v_3, v_4, v_2, v_6, v_5, v_8$)



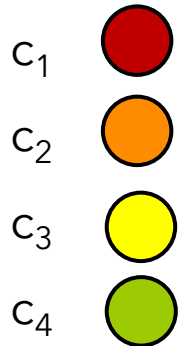
Άπληστος αλγόριθμος χρωματισμού των κορυφών του γράφου με τον ελάχιστο δυνατό αριθμό χρωμάτων ώστε γειτονικές κορυφές να μην έχουν το ίδιο χρώμα

Χρειάστηκαν **3 χρώματα**
(βέλτιστη λύση)

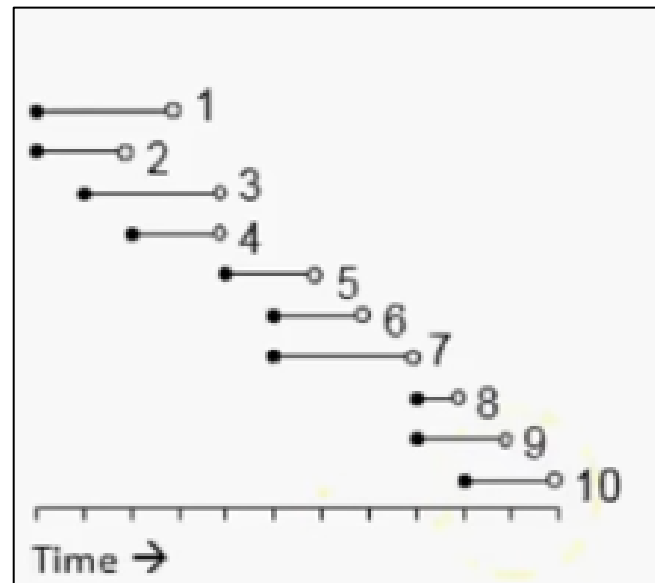
Θα μπορούσαμε με
2 χρώματα;



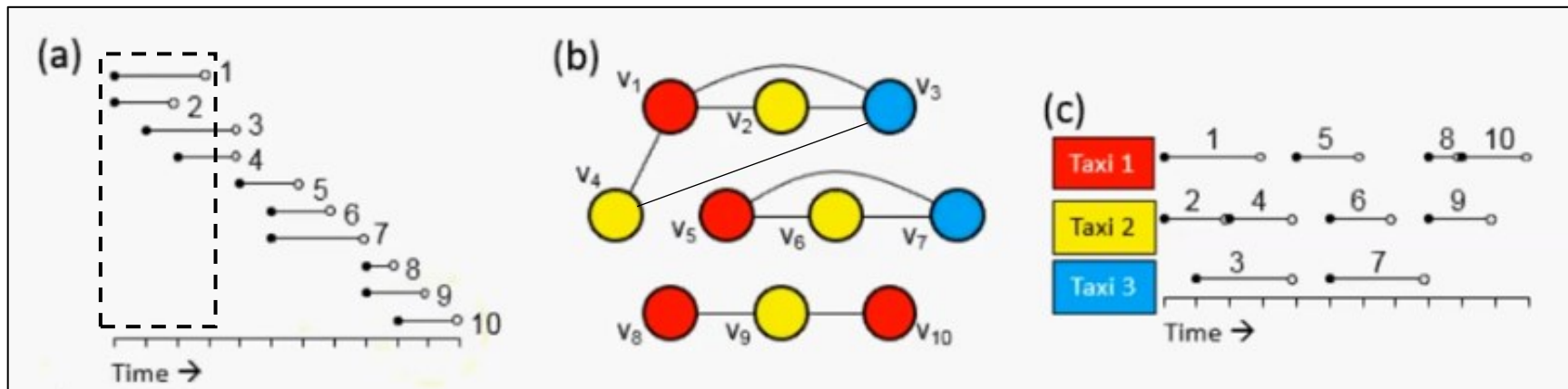
Τυχαία διάταξη κορυφών: ($v_1, v_6, v_7, v_5, v_2, v_3, v_4, v_8$)
διαφορετική



Μικρότερος αριθμός ταξί που θα καλύψουν τα ραντεβού



Μικρότερος αριθμός ταξί που θα καλύψουν τα ραντεβού



ίδιο χρώμα - 1 ταξί - 1 «κλάση»

ΣΥΝΕΧΕΙΑ ΣΤΟ ΕΠΟΜΕΝΟ ΦΡΟΝΤΙΣΤΗΡΙΟ...



Πηγές Φ7:

Σημειώσεις / διαλέξεις μαθήματος, προτεινόμενα βιβλία, υλικό μαθήματος προηγούμενων ετών,
SMA5503, Michael Sambol, Δ. Φωτάκης, Simply Explained